1 Java 1.0–1.4 · The foundations · cheat sheet

Java 1.0 – 1.4 · 1996 – 2002 · 16 features

Classes, interfaces, exceptions → Bytecode on a VM with GC → Look inside: reflection, inner classes → A real collections framework → HotSpot, regex, NIO

1 THE OBJECT MODEL

1 Classes and inheritance

One superclass, Object at the root

Single inheritance, Object at the root, virtual by default. final unless designed for extension; compose.

```
final class Circle extends Shape
```

2 Interfaces

A contract with no implementation

Multiple inheritance of type, none of state. Name the interface in signatures; keep each one small.

```
class A implements Runnable, Closeable
```

3 Exceptions

Failure modes in the signature

Error / RuntimeException (unchecked) / Exception (checked, in throws). finally always runs; chain the cause.

```
throw new MyEx("msg", cause);
```

2 THE MACHINE UNDERNEATH

4 JVM and bytecode

Write once, run anywhere (mostly)

javac → class files → any JVM. The class file is the contract; javap -c shows it. JNI (1.1) = native door.

```
javap -c Hello
```

5 Garbage collection

No free(), no dangling pointers

Managed heap; unreachable objects reclaimed. Leaks = forgotten references. No System.gc(); finalize() is gone.

```
java -Xlog:gc -Xmx512m App
```

6 HotSpot JIT

Interpret first, compile what is hot

Default VM since 1.3: interpret, profile, compile hot methods, inline, de-optimise. Warm up, then measure.

```
-XX:+PrintCompilation
```

7 Threads and synchronized

Monitors built into every object

Thread, Runnable, synchronized/wait/notify since 1.0. One lock for all shared state. start(), not run().

```
synchronized void inc() { n++; }
```

3 JAVA 1.1: LOOKING INSIDE

8 Inner and anonymous classes

A class inside a class, capturing it

Static nested, inner (hidden Outer.this), local, anonymous. Default to static; anonymous = old callbacks.

```
new Runnable() { public void run(){} }
```

9 Reflection and JavaBeans

Inspect and call types at run time

Class, Method, Field, Constructor at run time; JavaBeans get/set; dynamic proxies (1.3). Frameworks, not apps.

```
c.getMethod("setX", int.class)
```

10 Serialization

Objects to bytes, with a bill

Serializable + ObjectInput/OutputStream. Gadget-chain attacks: filter (JEP 290) or avoid; prefer JSON.

```
-Djdk.serialFilter=com.acme.*;!*
```

4 JAVA 1.2: COLLECTIONS

11 Collections framework

List, Set, Map: interfaces first

Collection/List/Set/Map interfaces; ArrayList/HashMap/TreeMap implementations. Declare the interface.

```
List<T> xs = new ArrayList<>();
```

12 Iterator, Comparator, utilities

Walk it, order it, wrap it

Iterator.remove() to delete safely; Comparable (natural) vs Comparator (external); Collections.sort etc.

```
Collections.sort(xs, cmp)
```

13 Vector, Hashtable, Enumeration

The 1.0 collections: legacy, avoid

~~DEPRECATED~~

Vector, Hashtable, Stack, Enumeration: locked per call, unsafe for check-then-act. Use ArrayList/HashMap.

```
Vector→ArrayList, Hashtable→HashMap
```

5 JAVA 1.4: THE TOOLBOX

14 Regular expressions

Compile once, match many

Pattern.compile once (static final); Matcher.find/matches/group. String.matches recompiles; (a+)+ backtracks.

```
Pattern.compile(re).matcher(s).find()
```

15 NIO buffers and channels

Buffers, channels, selectors

ByteBuffer (heap/direct) + Channel + Selector; flip() between write and read. NIO.2 (7) adds Files/Path.

```
buf.flip(); ... buf.clear();
```

16 assert and java.util.logging

Check invariants; log with levels

assert cond : msg needs -ea (off by default); java.util.logging levels/handlers. Validate with exceptions.

```
assert x > 0 : "x";  // java -ea
```

Do / don't, the headline features

Classes and inheritance
Do: final unless designed for extension; private fields; equals and hashCode together.
Don't: Don't extend to reuse code (Stack extends Vector exposed insertElementAt). Compose.

Exceptions
Do: Wrap with the original as cause; catch the most specific type you can handle.
Don't: No empty catch, no log-and-rethrow, never return from finally.

JVM and bytecode
Do: Spend ten minutes with javap -c: concat, boxing and synchronized become visible.
Don't: Don't assume behaviour is portable: paths, line endings, charset (UTF-8 default only since 18).

Threads and synchronized
Do: One lock for all shared state, reads included; prefer a private lock object.
Don't: start(), not run(); never lock on String literals or boxed values; Thread.stop throws.

Don't mix these up

Checked vs. unchecked exception
Checked extends **Exception** and must appear in `throws` or be caught. Unchecked extends **RuntimeException** (or Error) and may be ignored by the compiler, not by you.

Interface vs. abstract class
An interface is a **contract**; a class may implement many. An abstract class is a **partial class** with state and constructors; you get exactly one.

== vs. equals()
`==` compares **references** (same object). `equals()` compares **values** as the class defines them. Strings, boxed numbers, collections: always equals().

Inner vs. static nested class
An inner class holds a hidden **Outer.this** and cannot exist without an instance. A static nested class is **just a namespace**. Default to static.

Comparable vs. Comparator
`Comparable` is the type's **own** order (`compareTo` , one per class). `Comparator` is an order supplied **from outside**, as many as you like.

Vector vs. ArrayList
Same idea; `Vector` locks on **every call** and is still unsafe for check-then-act. `ArrayList` is unsynchronised and faster; use j.u.c when sharing.

start() vs. run()
`start()` asks the JVM for a **new thread**, which then calls run(). Calling `run()` yourself runs the body on the **current** thread, synchronously.

final vs. finally vs. finalize()
`final` : no reassignment, no override, no subclass. `finally` : runs on **every exit** from try. `finalize()` : a GC hook, deprecated, never use it.

Java 1.0 fixed the shape of the language: classes, interfaces, exceptions, a virtual machine with garbage collection, threads. The 1.x releases filled in the library: reflection, collections, a real JIT, NIO. Everything later stands on this floor.

Full cards: study guide, chapter 1 · knowledge graph · quiz · practice/

2 Java 5 · The language grows up · cheat sheet

Types that take parameters → Drop the daily boilerplate → Enums and annotations as types → Executors instead of threads → A memory model you can reason about

1 GENERICS

1 Generics — Typed collections, no casts

Type parameters on classes and methods, bounds with extends, inference at call sites. Never raw types.

```
<T extends Comparable<T>> T max(..)
```

2 Wildcards and PECS

? extends to read, ? super to write

List<Dog> is not a List<Animal>. ? extends T = read-only producer, ? super T = write-only consumer. PECS.

```
void f(List<? extends Number> xs)
```

3 Type erasure

Generics vanish after compilation

Type arguments erased to their bounds; one class per generic type. No new T(), T[] or instanceof List<T>.

```
List<String> → List  (in bytecode)
```

4 Covariant return types

Override with a narrower return type

An override may return a subtype of the parent's return type. Parameters stay invariant; @Override checks.

```
@Override Circle copy() { ... }
```

2 LESS CEREMONY

5 Enhanced for loop

for (T x : xs): no index, no Iterator

for (T x : arrayOrIterable). Implement Iterable to join in. Not for removal, not for index access.

```
for (Order o : orders) ...
```

6 Autoboxing and unboxing

int ↔ Integer, silently (and slowly)

Compiler inserts valueOf/intValue. Traps: == on Integer (cache -128..127), null unboxing NPE, remove(int).

```
Integer a = 128, b = 128; a == b // f
```

7 Varargs — Zero or more arguments, as T[]

T... last parameter = T[] inside. Powers format, asList, List.of. Allocates per call; asList(int[]) = 1 item.

```
static int sum(int... ns)
```

8 Static import — Math.max becomes max

import static pkg.Class.member; for constants and fluent DSLs (Collectors, Comparator, assertions).

```
import static java.lang.Math.max;
```

9 Scanner, printf, StringBuilder

Text in, text out, without the pain

Scanner (regex tokens), String.format/printf (%d %s %.2f %n), StringBuilder not StringBuffer, ProcessBuilder.

```
String.format("%,d %.2f%n", n, x)
```

3 ENUMS AND ANNOTATIONS

10 Enums — Constants that are real objects

Fixed instances with fields, constructors, methods, per-constant bodies. EnumSet/EnumMap. Persist name().

```
switch (s) { case OPEN -> ...; }
```

11 Annotations — Metadata that tools can read

@Override (always), @Deprecated, @SuppressWarnings (narrowly). Inert data until a tool or framework reads it.

```
@Override public String toString()
```

12 Writing annotations

Declare, retain, target, read

@interface Name { T elem() default ..; } + @Retention(RUNTIME) + @Target; read with getAnnotation.

```
m.getAnnotation(Audited.class)
```

4 CONCURRENCY

13 ExecutorService and Future

A pool owns the threads

ExecutorService pool; submit(Callable) → Future.get(timeout). Size pools on purpose; always shutdown().

```
Future<V> f = pool.submit(callable)
```

14 Concurrent collections and locks

Latches, locks, atomics, CHM

ConcurrentHashMap, CopyOnWriteArrayList, BlockingQueue, CountDownLatch, Semaphore, ReentrantLock, Atomic*.

```
latch.countDown(); latch.await();
```

15 Java Memory Model (JSR 133)

What one thread sees of another

Happens-before via volatile, locks, start/join. final fields publish safely. volatile is not atomic.

```
private volatile boolean stop;
```

16 JMX and java.lang.management

Look inside a running JVM

ManagementFactory MXBeans: memory, GC, threads, deadlocks, classes, OS. JMX remote + jconsole: secure it.

```
ManagementFactory.getMemoryMXBean()
```

Do / don't, the headline features

Generics

Do: Put the bound where the capability is used (`<T extends Comparable<? super T>>` for sorting) and let callers infer type arguments; if they must write them, the API is too clever.
Don't: Don't use raw types (`List` without `<>`): every check is lost and you get unchecked warnings. Don't overload on type arguments (`f(List<String>)` vs `f(List<Integer>)`): same erasure, won't compile.

Annotations

Do: Put `@Override` on every overriding method: it turns a silent overload (wrong parameter type, typo in the name) into a compile error.
Don't: Don't `@SuppressWarnings("unchecked")` a whole class; scope it to the one statement, with a comment. And don't expect an annotation to *do* anything by itself: it is inert until something reads it.

Enums

Do: Give enums behaviour (`status.terminal()`) instead of switching on them in five places, and use `EnumMap` / `EnumSet` whenever the key is an enum.
Don't: Don't persist `ordinal()` (reordering constants corrupts the data; store `name()` or an explicit code), and don't `switch` on an enum without `default` or exhaustiveness: a new constant then falls through silently.

ExecutorService and Future

Do: Size pools on purpose (CPU count for compute, more for blocking I/O), name threads through a `ThreadFactory` , always `shutdown()` (or try-with-resources, 19+), and always call `get` with a timeout.
Don't: Don't feed unbounded work to `newCachedThreadPool` (threads without limit), don't drop the `ExecutionException` that wraps the task's failure, and don't block inside a task waiting on another task in the same bounded pool: that deadlocks.

Don't mix these up

List<?> vs. List<Object>

`List<?>` is a list of **some unknown** type: any `List<X>` fits, and you can only read `Object` s. `List<Object>` accepts **only** a `List<Object>` .

? extends vs. ? super

`? extends T` : you **get** Ts out, cannot add. `? super T` : you can **put** Ts in, get only Object. Producer Extends, Consumer Super.

Integer == vs. equals()

`==` compares **references**; the valueOf cache makes -128..127 look equal and everything else not. `equals` compares the **value**. Or unbox first.

volatile vs. synchronized

`volatile` gives **visibility** of one variable, no atomicity. `synchronized` gives visibility **and** mutual exclusion over a whole block.

Runnable vs. Callable

`Runnable.run()` returns **nothing** and throws no checked exceptions. `Callable.call()` returns a **value** and may throw. Both go into `submit` .

execute() vs. submit()

`execute(Runnable)` is fire-and-forget; an exception goes to the **uncaught handler**. `submit` returns a **Future** that holds the exception until `get()` .

ordinal() vs. name()

`ordinal()` is the **position**, which changes when you reorder constants. `name()` is the **identifier**. Persist name() or an explicit code, never ordinal().

CLASS vs. RUNTIME retention

`CLASS` (the **default**) keeps the annotation in the class file but hides it from reflection. `RUNTIME` is what `getAnnotation` needs. `SOURCE` is for the compiler only.

One release, almost a new language: generics, enums, annotations, autoboxing, for-each, varargs, static imports, plus java.util.concurrent and a memory model that finally meant something. Most "modern Java" habits start here.

Full cards: study guide, chapter 2 · knowledge graph · quiz · practice/

3 Java 6–7 · Polish and plumbing · cheat sheet

Java 6 – 7 · 2006 – 2011 · 16 features

Embed the tools → Trim the ceremony → Close resources safely → Talk to the file system properly → Split work across cores → Prepare the bytecode

1 PROJECT COIN (JAVA 7)

1 Diamond operator

Infer the type arguments on new

new HashMap<>() infers the type arguments from the target. Not with anonymous classes until Java 9.

```
List<String> l = new ArrayList<>();
```

2 Strings in switch

case "GET": is finally legal

switch on a String selector; compiles to hashCode + equals, so it is case-sensitive. null throws NPE.

```
case "GET": return read(id);
```

3 try-with-resources

Close in reverse order, automatically

Resources in try (...) are closed in reverse order. Needs AutoCloseable; close failures become suppressed.

```
try (Reader r = open(p)) { ... }
```

4 Multi-catch and precise rethrow

One handler for several exceptions

catch (A | B e) shares one handler; e is final. Rethrowing e unchanged keeps the precise checked types.

```
catch (IOException | SQLException e)
```

5 Literals and @SafeVarargs

0b1010, 1_000_000 and quieter varargs

0b1010 and 1_000_000 literals; @SafeVarargs silences generic-varargs warnings on static/final methods.

```
0b1111_0000  ·  1_000_000L
```

2 JAVA 6: TOOLING AND APIS

6 Scripting and Compiler APIs

Run scripts and javac from Java code

javax.script (JSR 223): eval scripts via ScriptEngine. javax.tools (JSR 199): run javac in-process.

```
ToolProvider.getSystemJavaCompiler()
```

7 Pluggable annotation processing

Generate code at compile time

AbstractProcessor runs inside javac: read annotated elements via javax.lang.model, generate sources via Filer.

```
extends AbstractProcessor
```

8 Deque, ArrayDeque, NavigableMap

Both-ended queues and nearest-key maps

ArrayDeque replaces Stack/LinkedList as a deque. NavigableMap/Set: floorKey, ceilingKey, headMap, tailMap.

```
tiers.floorEntry(2_300)
```

9 Console, JDBC 4.0, ServiceLoader

Prompts, self-loading drivers, plugins

System.console().readPassword(); ServiceLoader.load(I.class) reads META-INF/services; JDBC drivers auto-load.

```
ServiceLoader.load(Codec.class)
```

3 JAVA 7 LIBRARIES

10 NIO.2 file system API

java.io.File, redone with real errors

Path + Files replace java.io.File: readAllLines, write, copy, move, walkFileTree; WatchService; async I/O.

```
Files.readAllLines(path, UTF_8)
```

11 Fork/Join framework

Split, recurse, join, steal idle work

ForkJoinPool + RecursiveTask/RecursiveAction: split below a threshold, fork, compute, join. Work stealing.

```
l.fork(); r.compute() + l.join()
```

12 Small java.util additions

Null-safe helpers, random, phased sync

Objects.equals/hash/requireNonNull; ThreadLocalRandom.current() per thread; Phaser = reusable barrier.

```
Objects.requireNonNull(x, "x")
```

13 Timsort as the default sort

Faster sort that checks your comparator

Stable Timsort for objects; a broken comparator throws "Comparison method violates its general contract".

```
Integer.compare(a, b), never a - b
```

4 RUNTIME

14 invokedynamic

A call site linked at run time

Instruction linked at first call via a bootstrap method + MethodHandle. Powers lambdas (8), string concat (9).

```
LambdaMetafactory.metafactory
```

15 G1 garbage collector

Regions, pause goals, concurrent marking

Region-based, concurrent-marking, pause-goal collector. -XX:+UseG1GC in 7u4+; default from 9; CMS gone in 14.

```
-XX:+UseG1GC -XX:MaxGCPauseMillis=100
```

16 Java 6 HotSpot performance

Cheaper locks, scalar-replaced objects

Escape analysis, lock elision/coarsening, biased locking: uncontended synchronized became nearly free.

```
-XX:+DoEscapeAnalysis
```

Do / don't, the headline features

try-with-resources

Do: Implement `AutoCloseable` on your own handle types and make `close()` idempotent; read `getSuppressed()` when a close failure matters (a failed commit on close is a real failure).

Don't: Don't open the resource before the `try` and only use it inside (it is not closed on failure), and don't catch and swallow exceptions inside `close()` to "keep it quiet".

Fork/Join framework

Do: Pick the sequential threshold by measuring (thousands of elements, not one), fork one half and `compute()` the other directly to save a task, and `join()` only after forking.

Don't: Don't block inside a task (the pool has about as many threads as cores; a blocked worker is a lost core), and don't `join()` before forking the sibling: that serialises the work.

NIO.2 file system API

Do: Use `Files.walkFileTree` (7) or `Files.walk` (8, inside try-with-resources) for trees; pass `ATOMIC_MOVE` when readers must never see a half-written file.

Don't: Don't poll a directory in a loop when `WatchService` exists, and don't ignore the `IOException` from `Files.delete`: unlike `File.delete()` it tells you why it failed.

invokedynamic

Do: Learn the signposts: `LambdaMetafactory` in a stack trace means a lambda, `StringConcatFactory` means `+` on strings; `-Djdk.invoke.LambdaMetafactory.dumpProxyClassFiles` shows the generated classes.

Don't: Don't be surprised that the first call of each lambda site is slower (linking), or that a lambda is not an anonymous class: there is no `.class` file and `getClass()` names are synthetic.

Don't mix these up

ArrayDeque vs. Stack

`Stack` is a **synchronised Vector** from 1.0 that iterates bottom-up. `ArrayDeque` is the deque to use: **push/pop at the head**, iterates top-down, no locks.

Suppressed vs. cause

A **cause** is what happened **before** (`initCause`, the chain). A **suppressed** exception happened **while closing** and was set aside; read it with `getSuppressed()`.

Multi-catch vs. catching the supertype

`catch (A | B e)` catches **only those two**. `catch (Exception e)` also swallows every **RuntimeException** you did not mean to handle.

floorKey vs. lowerKey

`floorKey` / `ceilingKey` are **inclusive** (≤ / ≥). `lowerKey` / `higherKey` are **strict** (< / >).

fork + join vs. compute vs. invoke

`fork()` **schedules** asynchronously, `join()` **waits** for it. `compute()` runs the work **inline**. `pool.invoke(task)` is how you start from **outside** the pool.

Path vs. File

`File` is both a **name** and a set of operations returning **booleans**. `Path` is just the name; `Files` holds the operations and throws **IOException** with a reason.

Diamond vs. raw type

`new ArrayList<>()` **infers** the type arguments. `new ArrayList()` is a **raw type**: a warning now, a ClassCastException later.

Annotation processor vs. reflection

A processor runs at **compile time** and **generates** files. Reflection runs at **run time** and **reads** annotations. Same `@Annotation`, different moment.

Mustang made the JDK programmable and faster; Dolphin, after five years, trimmed the syntax, fixed file I/O and laid the bytecode groundwork that lambdas would stand on three years later.

4 Java 8 · The functional turn · cheat sheet

Java 8 · Mar 2014 · LTS · 16 features

Pass behaviour as a value → Pipe data through it → Handle absence honestly → Upgrade the everyday APIs → Compose async work

1 FUNCTIONS AS VALUES

1 Lambda expressions

An anonymous function in one line

One-line implementation of a single-method interface. Types inferred from the target.

```
(a, b) -> a.length() - b.length()
```

2 Functional interfaces

The 43 shapes a lambda can take

Single-abstract-method interface = lambda target. Standard shapes live in java.util.function.

```
Function<A,B> · Predicate<T> · Supplier<T>
```

3 Method references

Point at a method instead of wrapping it

x -> foo(x) becomes Foo::bar. Static, bound, unbound and constructor forms.

```
map(String::trim)  ·  TreeSet::new
```

4 Default and static interface methods

Interfaces can ship a body now

Interface methods with bodies. How stream() and forEach() reached old collections.

```
default boolean isBig() { ... }
```

2 PIPELINES OVER DATA

5 Stream API

Lazy pipeline: source → ops → terminal

Lazy, one-shot pipeline. Intermediate ops chain; one terminal op runs it.

```
s.filter(p).map(f).collect(toList())
```

6 Collectors

Fold a stream into a result

toList, toSet, toMap, joining, groupingBy(+downstream), partitioningBy, counting.

```
groupingBy(Order::status, counting())
```

7 Parallel streams

Same pipeline, fork/join underneath

Splits work on the common fork/join pool. Only for big, CPU-bound, stateless pipelines.

```
list.parallelStream()  // measure first
```

8 Optional

"Might be absent" as a type

Return type that says "maybe absent". Chain map/flatMap/orElse; avoid get().

```
opt.map(User::city).orElse("?")
```

3 EVERYDAY API UPGRADES

9 java.time

Immutable dates that mean what they say

LocalDate/Time, Instant, ZonedDateTime, Duration, Period. Immutable. Replaces Date/Calendar.

```
LocalDate.now().plusDays(7)
```

10 Map helpers

The loops you kept rewriting

getOrDefault, putIfAbsent, computeIfAbsent, merge, forEach: the check-then-put loops, built in.

```
freq.merge(word, 1, Integer::sum)
```

11 Comparator combinators

Sort by this, then by that

comparing(key).thenComparing(...).reversed(); nullsFirst/Last; comparingInt avoids overflow.

```
comparing(P::last).thenComparing(P::first)
```

12 Strings, Base64 and friends

Small things that remove a utility class

String.join, StringJoiner, java.util.Base64, parallelSort, type and repeating annotations.

```
String.join(", ", list)
```

4 CONCURRENCY AND RUNTIME

13 CompletableFuture

Promises you can chain

Composable async future: thenApply, thenCompose, thenCombine, allOf, exceptionally. Pass an executor for blocking work.

```
supplyAsync(f, pool).thenApply(g)
```

14 Concurrency utilities update

Adders, StampedLock, ConcurrentHashMap 2.0

LongAdder/LongAccumulator, StampedLock, rewritten ConcurrentHashMap with bulk ops, commonPool().

```
LongAdder a; a.increment(); a.sum()
```

15 Metaspace replaces PermGen

Class metadata moved to native memory

Class metadata in native memory, auto-growing. PermGen flags ignored; cap with MaxMetaspaceSize.

```
-XX:MaxMetaspaceSize=512m
```

16 Nashorn JavaScript engine

JS on the JVM, deprecated 11, removed 15

~~REMOVED~~

JavaScript engine on invokedynamic. Deprecated 11, removed 15. Migration item only.

```
getEngineByName("nashorn")  // gone in 1
```

Do / don't, the headline features

Lambda expressions

Do: Keep lambdas to one expression or two or three lines. If it grows, give it a name: extract a method and use a method reference.

Don't: Don't mutate captured locals (they must be effectively final) and don't bury side effects in a lambda that looks like a pure function.

Optional

Do: Use it as a return type and resolve it quickly with `map` / `orElse*`. Use `orElseGet` when the fallback is expensive.

Don't: Don't use Optional for fields, parameters or collection elements, don't call `get()` without checking, and don't return `null` from a method typed Optional.

Stream API

Do: Keep each stage a single, side-effect-free step. Use `IntStream`, `LongStream`, `DoubleStream` for numeric work to avoid boxing.

Don't: Don't reuse a stream (it throws), don't mutate shared state inside `map` or `filter`, and don't force everything into a stream: a three-line loop with early return is often clearer.

java.time

Do: Store and transmit `Instant` (UTC); convert to a zone only for display or business rules that are zone-specific.

Don't: Don't use `Date`, `Calendar` or `SimpleDateFormat` in new code (mutable, zero-based months, not thread-safe). Don't use `LocalDateTime` for a moment that happened; it has no zone.

Don't mix these up

Lambda vs. method reference

A lambda **writes** the body inline. A method reference **points** at an existing method. Same target type either way.

map vs. flatMap

`map` turns each element into **one** element. `flatMap` turns each element into a **stream** and flattens. Same on Optional.

Intermediate vs. terminal

Intermediate ops (`filter`, `map`) are **lazy** and return a stream. The terminal op (`collect`, `count`) **runs** the pipeline, once.

orElse vs. orElseGet

`orElse(x)` evaluates `x` **always**. `orElseGet(supplier)` only when the Optional is **empty**.

thenApply vs. thenCompose

`thenApply` takes a function returning a **value**. `thenCompose` takes one returning a **CompletableFuture** and flattens it.

LocalDateTime vs. Instant

`LocalDateTime` is a **wall-clock reading** with no zone. `Instant` is a **point on the timeline** in UTC. Store Instants.

Default method vs. abstract class

A default method adds behaviour to an **interface** and allows multiple inheritance of behaviour. An abstract class can hold **state**; you get only one.

Collectors.toList() vs. Stream.toList()

`Collectors.toList()` (8) returns a **mutable** list of unspecified type. `Stream.toList()` (16) returns an **unmodifiable** list.

Functions became values, collections became pipelines, and dates finally made sense. Java 8 is the baseline every later version assumes you know.

Full cards: study guide, chapter 4 · knowledge graph · quiz · practice/

5 Java 9–10 · Modules and the new cadence · cheat sheet

Java 9 – 10 · 2017 – 2018 · 16 features

Modularise the JDK → Try things in a REPL → Write less, infer more → Finish the Java 8 APIs → Default to G1, know your container → Ship every six months

1 MODULES AND TOOLING

1 Java Platform Module System

module-info: requires, exports, opens

module-info.java: requires / exports / opens. Unnamed module = classpath; automatic = JAR on module path.

```
requires java.sql; exports shop.api;
```

2 JShell A REPL for Java, finally

jshell: evaluate Java statements interactively; /vars, /methods, /edit, /save, /open; API in jdk.jshell.

```
jshell> List.of(1, 2).size()
```

3 Multi-release JARs

One JAR, class files per Java version

Multi-Release: true + META-INF/versions/N/: alternative class files per Java version, chosen at run time.

```
jar --release 9 -C nine .
```

4 Version scheme and cadence

9.0.1, then a release every six months

JEP 223: 9.0.1, Runtime.version(). JEP 322 (10): March/September releases, LTS every 2-3 years. forRemoval.

```
Runtime.version().feature()
```

2 LANGUAGE AND EVERYDAY APIS

5 Local-variable type inference

var: the type is inferred, not dynamic

var x = expr; infers the static type of a local. Needs an initialiser; no fields, params, null or lambdas.

```
for (var e : map.entrySet())
```

6 Private interface methods

Share code between default methods

private (static) interface methods share code between defaults. Also: try (existingVar), diamond + anonymous.

```
private String join(String sep) {..}
```

7 Collection factory methods

List.of(1, 2, 3): immutable, no nulls

List.of / Set.of / Map.of: unmodifiable, null-hostile, duplicate-hostile; Set.of order random. copyOf (10).

```
Map.of("http", 80, "https", 443)
```

8 Stream and Optional additions

takeWhile, dropWhile, Optional.or(...)

Stream: takeWhile, dropWhile, iterate(seed, test, next), ofNullable. Optional: ifPresentOrElse, or, stream.

```
ids.map(f).flatMap(Optional::stream)
```

3 ASYNC AND OS APIS

9 Flow API and CompletableFuture

Back-pressure interfaces; CF timeouts

Flow: Publisher, Subscriber, Subscription, Processor. CF: orTimeout, completeOnTimeout, delayedExecutor.

```
cf.orTimeout(2, SECONDS)
```

10 Process API

pid, parent, children, onExit, destroy

ProcessHandle: current(), of(pid), children, info(), destroy, onExit() -> CompletableFuture; startPipeline.

```
ProcessHandle.current().pid()
```

11 HTTP/2 client (incubator)

jdk.incubator.httpclient, standard in 11

INCUBATOR

HTTP/2 + WebSocket client, sync and async (CompletableFuture). Incubator in 9/10; java.net.http in 11.

```
--add-modules jdk.incubator.httpclient
```

12 VarHandles and StackWalker

Unsafe replaced; lazy stack traces

VarHandle: typed, memory-ordered field/array access (Unsafe replacement). StackWalker: lazy frames, caller.

```
VH.compareAndSet(this, 0, 1)
```

4 RUNTIME

13 G1 as the default collector

Parallel GC out, G1 in; full GC parallel

G1 default on server-class machines (JEP 248); parallel full GC (JEP 307, 10). CMS deprecated 9, removed 14.

```
-Xlog:gc*   (G1 by default since 9)
```

14 Compact strings, indy concat

Latin-1 strings in half the memory

String stores byte[] + coder (Latin-1 or UTF-16): smaller heaps (254). a + b compiles to invokedynamic (280).

```
// "a" + b -> StringConcatFactory
```

15 Unified JVM logging

-Xlog: one flag for every subsystem

-Xlog:<tags>=<level>:<output>:<decorators>: one flag for GC, class loading, safepoints. Replaces -XX:+Print*.

```
-Xlog:gc*:file=gc.log:time,uptime
```

16 AppCDS and container awareness

Faster startup; cgroup CPU and memory

AppCDS (JEP 310): archive app classes for faster startup. cgroup-aware CPUs and heap; -XX:MaxRAMPercentage.

```
-XX:MaxRAMPercentage=75
```

Do / don't, the headline features

Java Platform Module System

Do: Start with your JARs as automatic modules and run `jdeps --jdk-internals`; use `--add-opens` only as a stopgap with a ticket attached.

Don't: Don't export everything to make errors go away, and don't rely on a name derived from a file name: set `Automatic-Module-Name`.

Local-variable type inference

Do: Follow the official style guide (openjdk.org/projects/amber/guides/lvti-style-guide): meaningful names, short scopes, an explicit type when the right-hand side hides it.

Don't: Don't use `var` with a diamond or a literal whose type surprises (`var n = 1` is `int`), and don't chain inference across several lines.

JShell

Do: Keep `jshell` open while coding; put imports and helpers in a `.jsh` file and `/open` it; add your classes with `--class-path`.

Don't: Don't paste a 60-line method into it to "test" it: write a unit test. Semicolons are optional at the top level, which trips up copy-paste back into a file.

Collection factory methods

Do: Use `List.of` for fixed data and `List.copyOf` to snapshot an argument; end pipelines you hand out with `toUnmodifiableList()` or `Stream.toList()` (16).

Don't: Don't use `Set.of` / `Map.of` where iteration order matters (it changes between runs), and don't store `null`: `Arrays.asList` tolerates it, `List.of` does not.

Don't mix these up

exports vs. opens

`exports` gives **compile-time** access to a package's **public** types. `opens` adds **run-time deep reflection** (private fields) for Hibernate, Jackson and friends.

Unnamed vs. automatic module

Classpath code lives in the **unnamed** module: reads everything, no name. A plain JAR on the **module path** becomes an **automatic** module, named by its manifest or file name, exporting every package.

List.of vs. Arrays.asList vs. unmodifiableList

`List.of`: **immutable copy**, no nulls. `Arrays.asList`: **fixed-size view** over the array, `set()` works, nulls fine. `Collections.unmodifiableList`: **read-only view** of a list that may still change underneath.

Optional.or vs. orElse vs. orElseGet

`or` returns an **Optional** (lazy). `orElse` returns the **value**, evaluating the fallback **always**. `orElseGet` returns the value, calling the supplier only when **empty**.

takeWhile vs. filter

`filter` tests **every** element and keeps the matches. `takeWhile` **stops at the first miss** and drops everything after it, even later matches.

var vs. dynamic typing

`var` is **compile-time inference**: the variable has one fixed static type, chosen from the initialiser. Nothing about the run time changed.

orTimeout vs. completeOnTimeout

`orTimeout` completes **exceptionally** with `TimeoutException`. `completeOnTimeout` completes **normally** with the default value you pass.

Incubator vs. preview

An **incubator** is an **API module** (`jdk.incubator.*`) enabled with `--add-modules`. A **preview** is a **language or VM feature** enabled with `--enable-preview` and tied to one release.

Java 9 reorganised the platform into modules and took three years doing it; Java 10 arrived six months later and changed how Java ships for good. In between: var, List.of, JShell and a long list of small library gifts.

Full cards: study guide, chapter 5 · knowledge graph · quiz · practice/

6 Java 11 · The new baseline · cheat sheet

Java 11 · Sep 2018 · LTS · 15 features

Replace HttpURLConnection → Polish the everyday APIs → Run a .java file directly → Observe and collect smarter → Drop Java EE, pick a vendor

1 EVERYDAY APIS

1 HTTP Client

HTTP/2, sync or async, no library

java.net.http: HttpClient (shared), HttpRequest, BodyHandlers.ofString; send() or sendAsync(); HTTP/2.

```
http.send(req, ofString())
```

2 New String methods

isBlank, lines, strip, repeat

isBlank(), lines(), strip()/stripLeading()/stripTrailing() (Unicode; trim() is ASCII), repeat(n).

```
s.lines().map(String::strip)
```

3 Files.readString and Path.of

A whole file in one call

Files.readString/writeString (UTF-8, OpenOptions), Path.of replaces Paths.get. Small files only.

```
Files.readString(Path.of("a.txt"))
```

4 Small API additions

isEmpty, Predicate.not, toArray(gen)

Optional.isEmpty(), Predicate.not(String::isBlank), list.toArray(String[]::new).

```
filter(not(String::isBlank))
```

2 LANGUAGE AND LAUNCHER

5 var in lambda parameters

var in lambdas, so you can annotate

(var a, var b) -> ... allowed; exists so lambda parameters can carry annotations. All or none.

```
(@NonNull var a, var b) -> a + b
```

6 Single-file source launcher

java Hello.java: no javac step

java Hello.java compiles in memory and runs; shebang scripts with --source. First class in the file runs.

```
java Hello.java  ·  #!/usr/bin/java
```

7 Nestmates and dynamic constants

No more access$ bridges; and condy

JEP 181: nested classes share privates natively (no access$ bridges). JEP 309: lazy constants (condy).

```
Outer.class.getNestMembers()
```

3 RUNTIME AND OBSERVABILITY

8 Flight Recorder

Always-on profiler, now in OpenJDK

JEP 328: open-sourced recorder, ~1% overhead. -XX:StartFlightRecording, jcmd JFR.*, jfr CLI, JMC viewer.

```
-XX:StartFlightRecording=duration=60s
```

9 Low-overhead heap profiling

Sampled allocations via JVMTI

JEP 331: JVMTI SampledObjectAlloc every ~512 KB; what profilers use for cheap allocation profiling.

```
SetHeapSamplingInterval(env, bytes)
```

10 ZGC Pauses under 10 ms, any heap size

EXPERIMENTAL

JEP 333: concurrent low-latency collector; experimental in 11 (Linux/x64), production 15, generational 21+.

```
-XX:+UseZGC  // +Unlock... on 11
```

11 Epsilon GC A collector that never collects

EXPERIMENTAL

JEP 318: no-op collector; OOM when the heap fills. For benchmarks, allocation measurement and tiny jobs.

```
-XX:+UseEpsilonGC  // test only
```

4 SECURITY AND REMOVALS

12 TLS 1.3 and new crypto

TLS 1.3, ChaCha20, Curve25519

TLS 1.3 in JSSE (JEP 332); ChaCha20-Poly1305 (329); X25519/X448 key agreement (324).

```
SSLContext.getInstance("TLSv1.3")
```

13 Java EE and CORBA removed

JAXB, JAX-WS, javax.annotation: gone

~~REMOVED~~

JEP 320: JAXB, JAX-WS, JTA, activation, CORBA, javax.annotation removed. Add jakarta.* jars. Migration pain.

```
jakarta.xml.bind-api + jaxb-runtime
```

14 Nashorn and Pack200 deprecated

Two more exits announced ~~DEPRECATED~~

JEP 335 Nashorn (removed 15), JEP 336 Pack200 (removed 14). Grep for jjs, nashorn, pack200 before upgrading.

```
@Deprecated(forRemoval = true)
```

15 Licensing and distributions

Oracle JDK paid; pick an OpenJDK build

Oracle JDK paid in production; free OpenJDK builds: Temurin, Corretto, Zulu, Microsoft. LTS 11, 17, 21, 25.

```
FROM eclipse-temurin:11-jre
```

Do / don't, the headline features

HTTP Client

Do: Build one `HttpClient` per application and share it (it owns the connection pool and threads). Set both `connectTimeout` and a per-request `timeout`.

Don't: Don't create a client per request, and don't forget that a 4xx or 5xx is a normal response, not an exception: check `statusCode()`.

Flight Recorder

Do: Leave recording on in production with the default settings (`settings=default`, about 1%); use `settings=profile` for short, targeted sessions.

Don't: Don't reach for a sampling agent or `jstack` loops before you have looked at a JFR file; and don't run `profile` settings for hours.

Single-file source launcher

Do: Keep `--source N` on the shebang line so the script pins its language level, and pass dependencies with `-cp`.

Don't: Don't ship production tools this way: the file is recompiled on every run and there is no caching. Until Java 22 (JEP 458) it also cannot reference a second source file.

ZGC

Do: Trial it on a staging copy with a realistic allocation rate; ZGC trades some throughput and memory for pause time.

Don't: Don't run the experimental 11 version in production, and don't expect it to fix an allocation storm: a collector that cannot keep up stalls allocation instead of pausing.

Don't mix these up

strip() vs. trim()

`trim()` removes characters ≤ U+0020 only. `strip()` removes anything `Character.isWhitespace` says is whitespace, **Unicode included**.

send vs. sendAsync

`send` **blocks** and returns the `HttpResponse`. `sendAsync` returns a **CompletableFuture** immediately; same request, same handler.

BodyHandler vs. BodyPublisher

A **handler** turns the **response** body into a String, file or stream. A **publisher** supplies the **request** body for POST and PUT.

Files.readString vs. Files.lines

`readString` loads the **whole file** into one String. `Files.lines` **streams** lazily and must be closed (try-with-resources).

Path.of vs. Paths.get

Same thing. `Path.of` (11) is the preferred spelling; `Paths.get` (7) stays for compatibility.

ZGC vs. Epsilon

`ZGC` is a real **low-pause** collector (experimental in 11). `Epsilon` **never collects**; it is a measuring tool that ends in an OOM.

Deprecated vs. deprecated for removal

Plain `@Deprecated` is **advice**. `forRemoval = true` is a **deadline**: Nashorn and Pack200 were removed within two releases.

Oracle JDK vs. Oracle OpenJDK build

Oracle JDK needs a **commercial licence** in production. Oracle's OpenJDK build (jdk.java.net) is **GPL** and free, like Temurin, Corretto and Zulu.

The first LTS of the six-month era. Few new keywords, but a real HTTP client, a scriptable launcher, two new collectors, and the removal of Java EE: the version every migration from 8 has to cross.

Full cards: study guide, chapter 6 · knowledge graph · quiz · practice/

7 Java 12–16 · Modern syntax arrives · cheat sheet

Java 12 – 16 · 2019 – 2021 · 16 features

Preview, feedback, finalise → Switch returns a value → Strings span lines → Data classes in one line → Test and bind in one step → Close the JDK internals

1 MODERN SYNTAX

1 Switch expressions

switch returns a value, no fall-through

case A, B -> value; blocks use yield; expression form must be exhaustive. Final in 14 (JEP 361).

```
case MON, TUE -> 8; default -> 0;
```

2 Text blocks

Multi-line strings that keep their shape

""" ... """; margin set by the closing delimiter; \ joins lines, \s keeps a space; .formatted(). Final in 15.

```
"""\n  SELECT 1\n  """.formatted(x)
```

3 Pattern matching for instanceof

Test and bind in one step

if (o instanceof String s) binds s; flow scoping works through && and negated early return. Final in 16.

```
if (!(o instanceof String s)) return;
```

2 DATA MODELLING, PREVIEWS

4 Records

A class that is just its fields

record P(int x, int y) {}: final fields, accessors, equals/hashCode/toString; compact ctor validates. In 16.

```
record Point(int x, int y) {}
```

5 Sealed classes

Only these subtypes; the compiler knows

PREVIEW

sealed interface S permits A, B; leaves final / sealed / non-sealed. Preview 15–16, final 17 (JEP 409).

```
sealed interface S permits A, B {}
```

6 Preview features

Try it now, compile with a flag

--enable-preview at javac and java; class files tied to one release. Incubator = jdk.incubator.* modules.

```
java --enable-preview --source 16 A
```

3 LIBRARY AND ERGONOMICS

7 Stream and Collector additions

toList(), mapMulti, teeing

Stream.toList() unmodifiable (16); mapMulti(BiConsumer) (16); Collectors.teeing(c1, c2, merge) (12).

```
teeing(minBy(c), maxBy(c), Pair::new)
```

8 String additions

indent, transform, formatted, strip

indent(n), transform(f) (12); formatted(args), stripIndent(), translateEscapes() (15).

```
"%s has %d".formatted(a, n)
```

9 Sockets redone, AF_UNIX channels

NIO under the old sockets; AF_UNIX

Socket/ServerSocket re-done on NIO (353), Datagram (373); UnixDomainSocketAddress with SocketChannel (380).

```
SocketChannel.open(UNIX).connect(addr)
```

10 Helpful NullPointerExceptions

The NPE says what was null

NPE message names the null sub-expression; opt-in flag in 14, default from 15; -g gives local names.

```
Cannot invoke "X.y()" because ...
```

4 RUNTIME AND JDK INTERNALS

11 ZGC and Shenandoah production

Two low-pause GCs, no unlock flag

ZGC (377) and Shenandoah (379) production in 15, no unlock flag. CMS removed in 14 (363): JVM won't start.

```
-XX:+UseZGC  ·  -XX:+UseShenandoahGC
```

12 JFR event streaming

Consume JFR events live, in process

RecordingStream: subscribe to JFR events in process (enable, onEvent, startAsync). Metrics without file dumps.

```
new RecordingStream().onEvent(n, h)
```

13 Strong encapsulation by default

--illegal-access=deny by default

16: --illegal-access=deny default; use --add-opens as a bridge. Hidden classes (15) for generated code.

```
--add-opens java.base/java.lang=...
```

5 TOOLING AND NATIVE

14 jpackage

Native installers from a jar

Native installers (msi, dmg, pkg, deb, rpm) with a bundled runtime via jlink. Incubator 14, final 16. Same OS.

```
jpackage --type dmg --main-jar a.jar
```

15 Foreign memory and linker

Off-heap memory and C calls without JNI

INCUBATOR

MemorySegment + scope, VarHandle access; CLinker downcalls. Incubator 14–16 (370, 383, 393, 389); final in 22.

```
MemorySegment.allocateNative(n, scope)
```

16 Vector API

SIMD loops that compile to AVX

INCUBATOR

jdk.incubator.vector: FloatVector, species, masks; JIT emits SIMD. Incubating since 16 (338), still in 25.

```
FloatVector.fromArray(S, a, i).mul(v)
```

Do / don't, the headline features

Switch expressions

Do: Switch over enums without `default` when you want the compiler to flag a new constant; add `default` only when the catch-all is genuinely the intent.

Don't: Don't mix arrow and colon labels in one switch (it does not compile), and don't use `yield` as an identifier any more; it is now a restricted keyword.

Pattern matching for instanceof

Do: Use the negated form with early return (`if (!(o instanceof T t)) return;`) to keep the happy path unindented.

Don't: Don't shadow a field or local with the binding name, and don't expect the binding after `||` : the compiler cannot prove it was matched.

Text blocks

Do: Put the closing `"""` on its own line, aligned with the content, so the block ends with a newline and the stripped margin is obvious.

Don't: Don't rely on the content's own indentation when the closing delimiter sits further left: it becomes part of the string. Mixed tabs and spaces are counted literally.

Records

Do: Validate in the compact constructor and keep components immutable: copy a `List` with `List.copyOf` there.

Don't: Don't add a mutable field through a static holder or expose a mutable array component unchanged; `equals` then lies. Don't use records for entities with identity (JPA needs a no-arg constructor and setters).

Don't mix these up

Switch expression vs. arrow label

Arrow (`->`) is a label form that never falls through; it works in **statements** too. An **expression** is a switch that **yields a value** and must be exhaustive.

yield vs. return

`yield` leaves a **switch block** with a value. `return` leaves the **method**. A `return` inside an expression switch does not compile.

Text block vs. stripIndent

A **text block** strips the margin at **compile time**, using the closing delimiter. `stripIndent()` does the same algorithm at **run time** on any String.

Record vs. class with final fields

A record **is** its components: equals, hashCode, toString and accessors are **derived**, and it cannot extend a class or add instance fields. A class only **promises** that.

Compact vs. canonical constructor

The **canonical** constructor repeats the parameter list and assigns fields. The **compact** one has **no parameter list**; fields are assigned after its body runs.

Preview vs. incubator

A **preview** is a **language or VM** feature behind `--enable-preview` . An **incubator** is an **API module** (`jdk.incubator.*`) you add with `--add-modules` .

Stream.toList() vs. Collectors.toList()

`Stream.toList()` (16) is **unmodifiable** and may hold nulls. `Collectors.toList()` (8) returns a list of **unspecified** type, in practice a mutable `ArrayList` .

--add-opens vs. --add-exports

`--add-exports` makes **public** types compile-time visible. `--add-opens` also allows **deep reflection** into privates; that is what old frameworks need on 16+.

Five releases in two years delivered the syntax Java had lacked for a decade: switch expressions, text blocks, records and instanceof patterns, each rehearsed as a preview before it became final.

Full cards: study guide, chapter 7 · knowledge graph · quiz · practice/

8 Java 17 · Sealed and settled · cheat sheet

Java 17 · Sep 2021 · LTS · 15 features

Close the hierarchy → Switch over every case → Lock the JDK internals → Drop the legacy → Settle on the new LTS

1 CLOSED HIERARCHIES

1 Sealed classes and interfaces

Only these subtypes, and no others

sealed ... permits A, B. Subtypes are final, sealed or non-sealed. Compiler knows every case.

```
sealed interface S permits A, B {}
```

2 Pattern matching for switch

Type patterns as case labels PREVIEW

case Type t [when guard] -> ... Exhaustive over sealed types. Preview 17, final in 21 (JEP 441).

```
case Circle c -> c.r()  // preview
```

3 ADT: records + sealed + switch

Data as a closed set of cases

Records (fields) + sealed interface (cases) + switch (dispatch) = enums with data, checked by the compiler.

```
sealed Result = Ok(v) | Err(msg)
```

2 LIBRARY ADDITIONS

4 Enhanced pseudo-random numbers

One interface, pluggable algorithms

RandomGenerator interface + RandomGeneratorFactory.of(name). LXM/Xoshiro, jumpable, splittable, nextInt(a, b).

```
RandomGeneratorFactory.of(name)
```

5 HexFormat **Bytes to hex and back, built in**

HexFormat.of().formatHex(bytes) / parseHex(s); ofDelimiter(":"), withUpperCase(), fromHexDigits.

```
HexFormat.of().formatHex(bytes)
```

6 Always-strict floating point

strictfp is now the only mode

IEEE 754 strict evaluation everywhere; strictfp is a no-op with a warning. Reproducible, not exact.

```
strictfp  // redundant since 17
```

7 Deserialization filters

Allow-list for readObject()

ObjectInputFilter per stream, per context or JVM-wide. Pattern allow-lists, limits, allow/reject/merge.

```
createFilter("com.acme.**;!*")
```

3 ENCAPSULATION AND REMOVALS

8 Strongly encapsulated internals

Internals locked; only --add-opens

Internal packages closed for good; --illegal-access removed. Only --add-opens module/pkg=target opens them.

```
--add-opens java.base/java.lang=...
```

9 Security Manager deprecated

The sandbox nobody configured

~~DEPRECATED~~

Security Manager deprecated for removal (warns on use); disabled for good in 24 (JEP 486). Isolate via the OS.

```
-Djava.security.manager  // warns
```

10 Removals: RMI Activation, jaotc

Three subsystems leave the JDK

~~REMOVED~~

RMI Activation and rmid removed; jaotc and Graal JIT removed (GraalVM lives on); Applet API deprecated.

```
java.rmi.activation  // gone
```

4 PLATFORM AND THE LTS

11 Upgrade 8/11 → 17: what breaks

The checklist every migration hits

Breaks: JAXB/EE modules, CMS, Nashorn gone; deep reflection errors; old ASM/Lombok/Mockito. Run jdeps first.

```
jdeps --jdk-internals app.jar
```

12 LTS every two years **Next LTS: 21, then 25**

LTS interval cut from three years to two: 17 (2021), 21 (2023), 25 (2025). Non-LTS: six months of updates.

```
--release 17  // baseline
```

13 macOS: Metal and AArch64 port

Apple silicon native; Metal for 2D

Metal rendering pipeline for Java 2D (default from 19) and a native macOS/AArch64 build for Apple silicon.

```
-Dsun.java2d.metal=true
```

14 Foreign Function & Memory API

Call C without JNI: first incubator

INCUBATOR

Incubator of Panama: MemorySegment, CLinker downcalls, SymbolLookup. Reshaped until final in 22 (JEP 454).

```
jdk.incubator.foreign  // 17 shape
```

15 Vector API (second incubator)

SIMD loops that become vector ops

INCUBATOR

jdk.incubator.vector: IntVector/FloatVector lanes compiled to SIMD. Still an incubator; waits for value types.

```
FloatVector.fromArray(S, a, i)
```

Do / don't, the headline features

Sealed classes and interfaces

Do: Keep the permitted types in the same file as the sealed type; then `permits` can be omitted and the file reads as one declaration.
Don't: Don't write `non-sealed` by reflex to "keep options open": it throws away exhaustiveness for every switch over the type. Default to `final` records.

Strongly encapsulated internals

Do: Run the test suite on 17 and collect every `InaccessibleObjectException`; upgrade the library first, add a targeted `--add-opens` only for what remains, and keep the list in one place (JVM args file or manifest).
Don't: Don't paste a blanket set of `--add-opens` lines from a blog post and move on: each one is technical debt that will break again when the internal moves. `sun.misc.Unsafe` is still reachable but on its own deprecation path.

ADT: records + sealed + switch

Do: Put the sealed interface and its records in one file, keep the records tiny, and write the operations as switches in the code that needs them.
Don't: Don't add a visitor interface on top; the switch is the visitor. Don't put `default -> throw` in these switches, it silences the compiler's missing-case check.

Upgrade 8/11 → 17: what breaks

Do: Compile with `--release 17`, run `jdeps --jdk-internals` on your jars, and run `jdeprscan`; both list exactly which internal APIs and removed modules you touch.
Don't: Don't start by changing source code; 90% of migration pain is in dependencies and JVM flags. And don't keep `-XX:+UseConcMarkSweepGC` or `MaxPermSize` in scripts: the JVM refuses to start.

Don't mix these up

sealed vs. final

`final` closes **one class**: no subclasses at all. `sealed` closes a **hierarchy**: only the listed subtypes, which may themselves be final, sealed or non-sealed.

non-sealed vs. an unlisted subclass

`non-sealed` is a **permitted** subtype that re-opens extension below it. A class **not in** `permits` cannot extend the sealed type at all; it is a compile error.

permits vs. same-file inference

`permits` names the subtypes **explicitly**, required when they live in other files. In **one file** (or one named module) you may omit it and the compiler infers the list.

--add-opens vs. --add-exports

`--add-exports` makes a package's **public** types visible at compile and run time. `--add-opens` also allows **deep reflection** on private members, which is what frameworks need.

--illegal-access vs. --add-opens

`--illegal-access=permit` opened **everything** at once and is removed in 17. `--add-opens` opens **one package to one module** and is the only way left.

Deprecated vs. deprecated for removal

`@Deprecated` alone is a **hint**: avoid in new code. `@Deprecated(forRemoval=true)` is a **schedule**: javac warns on every use and a later release deletes it (Security Manager, finalization, Applet).

RandomGenerator.of vs. RandomGeneratorFactory.of

`RandomGenerator.of(name)` returns a **generator** with a random seed. `RandomGeneratorFactory.of(name)` returns a **factory**: `create(seed)`, `isSplittable()`, `stateBits()`.

Incubator vs. preview

An **incubator** is an API in a `jdk.incubator.*` module that may change freely (FFM, Vector in 17). A **preview** is a complete language or API feature needing `--enable-preview`, expected to finalise with small changes (switch patterns).

Sealed types closed the data-modelling loop that records opened, the JDK locked its internals for good, and most teams made 17 their new baseline. This chapter is both what 17 added and what the modern toolkit now assumes.

Full cards: study guide, chapter 8 · knowledge graph · quiz · practice/

9 Java 18–20 · Loom and Panama in preview · cheat sheet

Java 18 – 20 · 2022 – 2023 · 15 features

Fix the defaults → Preview virtual threads → Structure concurrency → Deconstruct records → Reach native memory → Finalise in 21

1 LOOM: THREADS AND STRUCTURE

1 Virtual threads

A thread per task, millions of them

PREVIEW

Thread.ofVirtual(), newVirtualThreadPerTaskExecutor(): JDK-scheduled threads, blocking is cheap. Final in 21.

```
newVirtualThreadPerTaskExecutor()
```

2 Structured concurrency

Subtasks die with their parent INCUBATOR

StructuredTaskScope: fork subtasks, join, propagate failure, cancel the rest on exit. Incubator 19/20.

```
scope.fork(task); scope.join()
```

3 Scoped values

Immutable, inherited context

INCUBATOR

ScopedValue.where(KEY, v).run(...): immutable, inherited by structured subtasks, no copies. Final 25.

```
ScopedValue.where(USER, u).run(r)
```

2 LANGUAGE PREVIEWS

4 Record patterns

Deconstruct where you test PREVIEW

instanceof Point(int x, int y), nested Line(Point(..), Point(..)), var components. Preview 19/20, final 21.

```
if (o instanceof Point(var x, var y))
```

5 Pattern matching for switch

Guards, null cases, dominance rules

PREVIEW

case T t when cond, case null, dominance checks, MatchException for stale classes. Previews 2–4; final 21.

```
case Integer i when i > 100 -> ...
```

6 Preview and incubator lifecycle

Why three unused releases matter

Preview: complete, --enable-preview, class files tied to one release. Incubator: jdk.incubator.*, unstable.

```
javac --enable-preview --release 20
```

3 PANAMA AND PORTS

7 Foreign Function & Memory API

strlen from Java in six lines, no JNI

PREVIEW

Arena, MemorySegment, Linker.nativeLinker().downcallHandle(...). Incubator 18, preview 19/20, final 22.

```
try (Arena a = Arena.ofConfined())
```

8 Vector API (3rd–5th incubator)

Still incubating; FFM segments

INCUBATOR

jdk.incubator.vector keeps incubating (417, 426, 438): masks, compress/expand, load/store via MemorySegment.

```
IntVector.fromMemorySegment(S, seg, o)
```

9 Linux/RISC-V port

HotSpot on an open instruction set

HotSpot (interpreter, C1, C2, all GCs) on RV64 Linux; one more os.arch value to respect in native loaders.

```
os.arch = riscv64
```

4 JAVA 18: DEFAULTS AND TOOLING

10 UTF-8 by default

UTF-8 on every OS, finally

Default charset is UTF-8 on every OS (file.encoding). COMPAT restores the old one; native.encoding reports it.

```
-Dfile.encoding=COMPAT  // opt out
```

11 Simple web server

jwebserver: static files, zero config

jwebserver -p 8000 -d dir serves static files; SimpleFileServer / HttpHandlers API for tests. GET/HEAD only.

```
jwebserver -p 9000 -d ./docs
```

12 Code snippets in Javadoc

@snippet: examples javac checks

{@snippet lang=java : ...} or {@snippet file= region=}; @highlight/@replace/@link; external examples compile.

```
{@snippet file="Demo.java" region="x"}
```

5 JAVA 18: UNDER THE HOOD

13 Reflection via method handles

Method.invoke on method handles

Method.invoke / Field.get run on method handles; native and generated accessors removed. Stack frames differ.

```
DirectMethodHandleAccessor in traces
```

14 Internet-address resolution SPI

Pluggable DNS for InetAddress

InetAddressResolverProvider SPI: your own resolver (DoH, discovery, tests) replaces the OS call. One per JVM.

```
extends InetAddressResolverProvider
```

15 Finalization deprecated

finalize() is on its way out ~~DEPRECATED~~

finalize() deprecated for removal (--finalization=disabled to test). Use try-with-resources + Cleaner.

```
cleaner.register(this, staticAction)
```

Do / don't, the headline features

Virtual threads

Do: Create a new virtual thread per task and never pool them; let `newVirtualThreadPerTaskExecutor` do it. Replace `synchronized` around blocking I/O with `ReentrantLock` on 19 to 23 to avoid **pinning** the carrier (fixed in 24, JEP 491).
Don't: Don't cache expensive objects in `ThreadLocal` (a million threads means a million copies) and don't call `Object.wait` or native code while holding a monitor in a hot path on these releases.

Foreign Function & Memory API

Do: Use `Arena.ofConfined()` in try-with-resources for request-scoped memory and `invokeExact` with exact types; generate bindings with `jextract` instead of writing descriptors by hand.
Don't: Don't let a `MemorySegment` escape its arena (it throws `IllegalStateException` after close, which is the point) and don't use `Arena.global()` for anything you ever want freed.

Record patterns

Do: Use `var` for the components to keep patterns short; the types are known from the record. Nest patterns instead of chaining `instanceof` and accessor calls.
Don't: Don't deconstruct when you need the whole record too; bind the record (`case Point p`) and call accessors, or you end up rebuilding it.

UTF-8 by default

Do: Test the upgrade with `-Dfile.encoding=UTF-8` on 17 first: if that breaks, 18 breaks the same way. Then fix by naming the real charset of each legacy file.
Don't: Don't paper over it with `COMPAT` in production; it keeps the behaviour that differs per machine. And don't assume the console is UTF-8: on Windows `System.out` may still write a code page.

Don't mix these up

Virtual thread vs. carrier thread

A **virtual** thread is the task's thread, cheap and heap-allocated. A **carrier** is one of the few platform threads it is mounted on while running; blocking unmounts it, pinning does not.

Pooling vs. one thread per task

Platform threads are expensive, so you **pool** them. Virtual threads are cheap, so you create **one per task** and never reuse; a pool of virtual threads defeats the purpose.

ThreadLocal vs. ScopedValue

`ThreadLocal` is **mutable**, lives as long as the thread and is **copied** to child threads. `ScopedValue` is **immutable** within a bounded region and is **shared** with structured subtasks.

ShutdownOnFailure vs. ShutdownOnSuccess

`ShutdownOnFailure` : all subtasks must succeed; the **first failure** cancels the rest. `ShutdownOnSuccess` : take the **first result**, cancel the rest (hedged requests).

Type pattern vs. record pattern

`case Point p` is a **type** pattern: it binds the whole object. `case Point(var x, var y)` is a **record** pattern: it binds the components and can nest.

Guard vs. dominance

A **guard** (`when`) narrows a case at run time. **Dominance** is a compile-time rule: a case that an earlier case already covers entirely is an error, so guarded cases go first.

Preview vs. incubator

Preview: a finished language or API feature behind `--enable-preview` , likely to finalise soon. **Incubator**: a `jdk.incubator.*` module that may change or disappear. Virtual threads previewed; the Vector API incubates.

file.encoding vs. native.encoding

`file.encoding` is the **default charset** for files and strings, UTF-8 since 18. `native.encoding` (new in 18) reports the **OS locale** charset, which `System.out` still uses on the console.

Nobody deploys 18, 19 or 20, yet they are where Java 21 was designed: virtual threads, record patterns, switch patterns and the foreign API all went through their preview rounds here while 18 quietly fixed defaults nobody wanted to think about.

Full cards: study guide, chapter 9 · knowledge graph · quiz · practice/

10 Java 21 · Virtual threads and patterns · cheat sheet

Java 21 · Sep 2023 · LTS · 16 features

Block cheaply on a virtual thread → Take data apart with patterns → Let sealed types make switch exhaustive → Ask any ordered collection for first and last → Watch the previews: some change, some vanish

1 THREADS THAT SCALE

1 Virtual threads

Cheap threads: block freely, scale wide

JVM-scheduled threads parked on the heap when blocked. One per task, never pooled; for I/O-bound work.

```
Thread.ofVirtual().start(task)
```

2 Structured concurrency

Fork tasks that live inside one scope PREVIEW

StructuredTaskScope: fork, join, read; leaving the scope cancels stragglers. Preview until 25.

```
scope.fork(() -> findUser(id))
```

3 Scoped values

Read-only context for a call tree PREVIEW

ScopedValue.where(K, v).run(...): immutable context for a call tree, cheap to inherit. Final in 25.

```
ScopedValue.where(USER, u).run(r)
```

2 PATTERNS AND DATA

4 Record patterns

Deconstruct a record where you test it

o instanceof Point(int x, int y) binds the components; patterns nest; also as switch cases.

```
o instanceof Point(var x, var y)
```

5 Pattern matching for switch

switch on type, with guards and null

case Type t when guard -> ...; case null; exhaustive over sealed types without default.

```
case Circle c when c.r() > 9 -> ...
```

6 Data-oriented programming

Records + sealed + switch = ADTs

Records (values) + sealed (closed set) + switch patterns (exhaustive ops): algebraic data types in Java.

```
sealed interface R permits Ok, Err {}
```

7 Unnamed patterns and variables

_ for the parts you don't care about PREVIEW

Underscore for unused variables, catch parameters and pattern components; reusable in one scope. Final in 22.

```
case Line(Point(var x, _), _) ->
```

3 SYNTAX AND STRINGS

8 Unnamed classes, instance main

Hello world without the ceremony PREVIEW

No class declaration, void main() as an instance method. Preview 21–24, final in 25 as compact source files.

```
void main() { ... }  // no class
```

9 String templates

STR."Hi \{name}": tried, then dropped ~~WITHDRAWN~~

Embedded \{expr} with a template processor. Preview 21–22, withdrawn before 23. Use formatted().

```
STR."Hi \{name}"  // 21–22 only
```

10 Unicode 15 and small APIs

repeat, indexOf(ch, from, to), clamp

StringBuilder.repeat, indexOf(ch, from, to), Math.clamp, Character.isEmoji, HttpClient.close(), Unicode 15.

```
Math.clamp(v, 0, 100)
```

4 LIBRARY AND NATIVE

11 Sequenced collections

getFirst, getLast, reversed on any order

SequencedCollection/Set/Map: getFirst/getLast, addFirst/Last, removeFirst/Last, reversed() view.

```
list.getLast()  ·  set.reversed()
```

12 Key Encapsulation Mechanism API

Key exchange for the post-quantum era

javax.crypto.KEM: encapsulate/decapsulate a shared secret. The shape ML-KEM plugs into (Java 24).

```
KEM.getInstance("DHKEM")
```

13 FFM API, third preview

Call C without JNI: last preview PREVIEW

Linker + MemorySegment + Arena: call C and manage off-heap memory with no JNI. Third preview; final in 22.

```
Arena.ofConfined()  ·  downcallHandle
```

5 RUNTIME AND PLATFORM

14 Generational ZGC

ZGC learns young and old generations

-XX:+ZGenerational: ZGC with a young generation. Default in 23, the only mode in 24.

```
-XX:+UseZGC -XX:+ZGenerational
```

15 Agent and 32-bit port warnings

Attach-time agents warn; Win32 goes

Attaching an agent at run time warns; use -XX:+EnableDynamicAgentLoading or -javaagent. Win32 deprecated.

```
-XX:+EnableDynamicAgentLoading
```

16 Vector API, sixth incubator

SIMD from Java, still incubating INCUBATOR

jdk.incubator.vector: explicit SIMD. Sixth incubator; waits for Valhalla value types.

```
FloatVector.fromArray(S, a, i)
```

Do / don't, the headline features

Virtual threads

Do: Create one virtual thread per task with `newVirtualThreadPerTaskExecutor()` and let it die; keep CPU-heavy work on a bounded platform pool.

Don't: Don't pool virtual threads, don't guard I/O with `synchronized` on 21 (it pins the carrier; fixed in 24, use `ReentrantLock` meanwhile), and don't cache big objects in `ThreadLocal` when a million threads may be alive.

Pattern matching for switch

Do: Make the selector type sealed and leave out `default`, so the compiler tells you when a new case appears.

Don't: Don't add `default` to a sealed switch out of habit (it silences exhaustiveness checking), and don't put the broad pattern before the narrow one: the compiler rejects a dominated case.

Record patterns

Do: Use `var` for component types in nested patterns; the compiler already knows them and the line stays readable.

Don't: Don't deconstruct a record you then need whole: bind it as `case Line l` instead. Names in the pattern are new variables, not the record's component names.

Sequenced collections

Do: Write `getFirst()` / `getLast()` and iterate `reversed()`; it reads better and works the same on every ordered collection.

Don't: Don't call `addFirst` on an immutable `List.of` (UnsupportedOperationException), and don't expect `reversed()` to be a copy: changes show through both ways. `LinkedHashSet.addFirst` moves an element that is already present.

Don't mix these up

Virtual vs. platform thread

A **platform** thread wraps one OS thread; blocking it wastes it. A **virtual** thread is parked on the heap when it blocks and costs almost nothing. Same `Thread` API.

Pooling vs. one per task

Pools exist because platform threads are **expensive**. Virtual threads are **cheap**: create one per task, never pool them. Limit concurrency with a `Semaphore` instead.

Blocking vs. pinning

Blocking parks the virtual thread and frees the carrier. **Pinning** (a `synchronized` block or native frame on 21) keeps the carrier stuck. Fixed for `synchronized` in 24.

Type pattern vs. record pattern

`case Point p` binds the **whole** object. `case Point(int x, int y)` **deconstructs** it into components. Both can carry a `when` guard.

when guard vs. if inside the case

A `when` guard is part of the **case**, so a non-matching value falls through to the next case. An `if` in the body **consumes** the case and must handle the else itself.

case null vs. no case null

With `case null` the switch **handles** null. Without it a null selector throws **NullPointerException**, even if there is a `default`.

ScopedValue vs. ThreadLocal

`ThreadLocal` is **mutable**, lives as long as the thread and is copied to children. `ScopedValue` is **immutable**, scoped to a `run` block and shared with children by reference.

reversed() vs. Collections.reverse()

`list.reversed()` returns a live **view**; writes go through. `Collections.reverse(list)` reorders the list **in place**. Neither copies.

Blocking code finally scales, and data finally has a shape the compiler can check. Java 21 is the LTS that makes Loom and pattern matching everyday tools.

Full cards: study guide, chapter 10 · knowledge graph · quiz · practice/

11 Java 22–24 · Gatherers, FFM and simpler starts · cheat sheet

Java 22 – 24 · 2024 – 2025 · 16 features

Call C through the FFM API → Write your own stream stage → Run a folder, document in Markdown → Cache the startup, shrink the headers → Retire Unsafe, the Security Manager and 32-bit

1 LANGUAGE

1 Unnamed variables and patterns

_ is final: skip what you don't read

Underscore for unused variables and pattern components, final. Reusable within one scope.

```
case Line(Point(var x, _), _) ->
```

2 Primitive types in patterns

case int i when i > 0, instanceof byte

PREVIEW

instanceof and case patterns for int, long, byte, double, boolean; exact-fit narrowing. Still preview.

```
case int i when i < 10 -> ...
```

3 Flexible constructor bodies

Validate before you call super(...) PREVIEW

Statements (and from 23 field assignments) before super()/this(). Preview 22–24; final in 25.

```
if (v <= 0) throw ...; super();
```

4 Compact files & module imports

A program is a file with a main method

PREVIEW

void main() without a class; import module java.base; one line for a module's packages. Final in 25.

```
import module java.base;
```

5 String templates withdrawn

Two previews, then gone without a trace

~~WITHDRAWN~~

STR."...\{x}" previewed in 21 and 22, removed from 23. Previews can disappear; use formatted().

```
STR."Hi \{x}"  // 23+: no such thing
```

2 STREAMS, BYTECODE AND NATIVE

6 Stream gatherers

Write your own intermediate operation

Stream.gather(g): custom intermediate ops. Built-ins: windowFixed, windowSliding, fold, scan, mapConcurrent.

```
s.gather(Gatherers.windowSliding(2))
```

7 Class-File API

Parse and generate bytecode, no ASM

java.lang.classfile: parse, transform and build class files with the JDK's own API. Final in 24.

```
ClassFile.of().parse(bytes)
```

8 Foreign Function & Memory API

Call C and own off-heap memory, no JNI

Linker + Arena + MemorySegment: native calls and off-heap memory without JNI. Enable native access explicitly.

```
try (Arena a = Arena.ofConfined())
```

3 RUNTIME, MEMORY AND GC

9 AOT class loading & linking

Project Leyden begins: cache the startup

-XX:AOTMode=record / create, then -XX:AOTCache=app.aot: pre-loaded, pre-linked classes. Leyden step one.

```
java -XX:AOTCache=app.aot -jar app.jar
```

10 Compact object headers

Object headers go from 96 to 64 bits

EXPERIMENTAL

-XX:+UseCompactObjectHeaders: 64-bit headers, 10–20% less heap for small objects. Experimental 24, product 25.

```
-XX:+UseCompactObjectHeaders
```

11 G1, ZGC and Shenandoah updates

Generational everywhere, G1 trimmed

G1 region pinning (22), generational ZGC default (23), generational Shenandoah and late G1 barriers (24).

```
-XX:ShenandoahGCMode=generational
```

12 Virtual threads: no more pinning

synchronized stops pinning the carrier

synchronized no longer pins virtual threads to carriers. Native frames and class init still do.

```
// 24: synchronized is Loom-friendly
```

4 SECURITY, INTEGRITY AND TOOLING

13 Unsafe and JNI warnings

Unsafe memory access and JNI now warn

~~DEPRECATED~~

Unsafe memory methods deprecated (23) and warn (24); JNI warns without --enable-native-access (24).

```
--enable-native-access=ALL-UNNAMED
```

14 Post-quantum crypto and KDF API

ML-KEM, ML-DSA, and a KDF API

ML-KEM (via KEM) and ML-DSA (via Signature) final in 24; KDF API preview in 24, final in 25.

```
KEM.getInstance("ML-KEM")
```

15 Security Manager, 32-bit retired

Security Manager off for good; Win32 out

~~REMOVED~~

Security Manager permanently disabled (setter throws); Windows 32-bit removed, Linux 32-bit deprecated.

```
setSecurityManager(sm)  // throws
```

16 Launcher, Javadoc and jlink

Run a folder; /// docs; jlink sans JMODs

java Main.java compiles the folder (22); /// Markdown Javadoc (23); jlink without JMODs on linkable JDKs (24).

```
/// Markdown **doc** with [links]
```

Do / don't, the headline features

Unnamed variables and patterns

Do: Replace every `ignored`, `unused` and `e`-that-is-never-logged with `_` as you touch the code; the compiler and the reader both stop asking.

Don't: Don't use `_` where you still read the value a line later: it is not an identifier and the compiler rejects the read.

Stream gatherers

Do: Reach for the built-ins first (`windowSliding`, `scan`); write `Gatherer.ofSequential` with a tiny mutable state object when you need your own. Return `false` from the integrator to stop early.

Don't: Don't shoehorn a gatherer in where `map` or `filter` already does the job, and don't share mutable state between parallel gatherers unless you also provide a combiner.

Compact files & module imports

Do: Pair it with `java Hello.java` (no compile step) and move to real classes and packages when the program needs structure.

Don't: Don't mix module imports from modules that export the same simple name without a single-type import to disambiguate: the compiler reports the ambiguity, not a silent pick.

Foreign Function & Memory API

Do: Generate bindings with `jextract`, pass `--enable-native-access=ALL-UNNAMED` (or your module) on the command line, and close arenas in try-with-resources.

Don't: Don't use `Arena.global()` for anything with a lifetime, don't touch a segment after its arena closed, and don't share a confined arena's segments across threads (use `ofShared`).

Don't mix these up

Gatherer vs. Collector

A **Collector** ends the stream and produces one result (`collect`). A **Gatherer** sits in the middle and produces another stream (`gather`). Both carry state.

windowFixed vs. windowSliding

`windowFixed(3)` cuts **disjoint** chunks: [1,2,3],[4,5,6]. `windowSliding(3)` **steps by one**: [1,2,3],[2,3,4],[3,4,5]. Neither pads the last window.

fold vs. scan

`fold` emits **one** element at the end (like `reduce` with a seed). `scan` emits every **intermediate** result, so n elements in, n out.

Arena.ofConfined vs. ofShared

Confined: one thread owns the memory, fastest, closes deterministically. **Shared**: any thread may access it, closing is a safepoint operation. `ofAuto` is GC-managed; `global` never frees.

ML-KEM vs. ML-DSA

ML-KEM is key **encapsulation** (agree a secret, via the `KEM` API). **ML-DSA** is a **signature** scheme (prove authorship, via `Signature`). Both lattice-based, both FIPS.

Compact source file vs. single-file launch

Single-file launch (Java 11) is how you **run** a source file. A **compact source file** (preview 21–24, final 25) is how you **write** it: no class, `void main()`.

AOT cache vs. CDS archive

CDS stores parsed class metadata. The **AOT cache** (JEP 483) builds on it and also stores classes **loaded and linked**, so startup skips more. Same two-step record/create idea.

Prologue vs. instance initializer

A constructor **prologue** runs **before** `super()` and may not touch `this`. An instance initializer block runs **after** `super()`, when the object exists.

Three non-LTS releases that finished Panama, gave streams a missing operation, started Project Leyden and closed the JVM's back doors. Most of what previewed here goes final in 25.

Full cards: study guide, chapter 11 · knowledge graph · quiz · practice/

12 Java 25 · The new LTS · cheat sheet

Java 25 · Sep 2025 · LTS · 16 features

Write less ceremony → Share context on virtual threads → Defer work safely → Start and warm up faster → Profile and secure in production → Plan the 21 → 25 jump

1 LANGUAGE, FINALISED

1 Compact source files & main()

A program is a file with a main()

Fields and methods without a class; void main(); IO.println; implicit import module java.base. Final.

```
void main() { IO.println("hi"); }
```

2 Module import declarations

One import for a whole module

import module java.base; brings every exported package on demand; a clash is fixed by a single-type import.

```
import module java.base;
```

3 Flexible constructor bodies

Statements before super(...)

Validate, compute and assign your own fields before super(...)/this(...); no use of this in the prologue.

```
check(x); this.f = x; super(x);
```

4 Primitive types in patterns

instanceof int, switch on long PREVIEW

Preview: instanceof byte b; switch on long/float/double/boolean; a pattern matches only an exact conversion.

```
case int i when i > 0 -> ...
```

2 CONCURRENCY AND LAZY STATE

5 Scoped values

Immutable context, scoped to a call

where(KEY, v).run(...) binds for the call; callees get(); inherited by StructuredTaskScope children. Final.

```
where(USER, u).run(() -> work())
```

6 Structured concurrency

Fork, join, cancel as one unit PREVIEW

Preview. StructuredTaskScope.open(joiner); fork → Subtask; join(); a failure cancels siblings. New API in 25.

```
open(Joiner.allSuccessfulOrThrow())
```

7 Stable Values

Lazy once, then treated as a constant

PREVIEW

Preview. StableValue.of() + orElseSet(supplier): set once, constant-folded; list/map forms. LazyConstant in 26

```
LOG.orElseSet(() -> Logger.create())
```

3 STARTUP, MEMORY AND SIMD

8 AOT cache ergonomics & profiles

Train once, start fast, warm up faster

-XX:AOTCacheOutput=app.aot trains and writes the cache in one run; cached method profiles shorten warm-up.

```
-XX:AOTCacheOutput=app.aot
```

9 Compact object headers

Every object 4 bytes smaller

64-bit headers instead of 96: smaller heap, fewer GCs. Opt in: -XX:+UseCompactObjectHeaders; default in 27.

```
-XX:+UseCompactObjectHeaders
```

10 Generational Shenandoah

Shenandoah learns young and old

-XX:+UseShenandoahGC -XX:ShenandoahGCMode=generational is production-ready; the default mode is unchanged.

```
-XX:ShenandoahGCMode=generational
```

11 Vector API SIMD loops, still incubating

INCUBATOR

Incubator since 16, waiting for Valhalla. FloatVector.fromArray(S, a, i).mul(...).intoArray(c, i).

```
FloatVector.fromArray(S, a, i)
```

4 SECURITY, OBSERVABILITY, PLATFORM

12 Key Derivation Function API

HKDF as a first-class JCA service

javax.crypto.KDF with HKDF-SHA256/384/512; HKDFParameterSpec.ofExtract().addIKM(k).thenExpand(info, n).

```
KDF.getInstance("HKDF-SHA256")
```

13 PEM encoding API

Keys and certs to and from PEM text

PREVIEW

Preview. PEMEncoder.of().encodeToString(key); PEMDecoder.of().decode(text, PrivateKey.class). Final in 28.

```
PEMEncoder.of().encodeToString(key)
```

14 JFR: CPU-time, sampling, tracing

Profile production without crashing it

518 safe sampling; 509 jdk.CPUTimeSample (experimental, Linux); 520 method-timing= and method-trace= filters.

```
jcmd PID JFR.start method-timing=C::m
```

15 32-bit x86 port removed

HotSpot is 64-bit only on x86 ~~REMOVED~~

Linux x86-32 port removed (Windows x86-32 went in 24). Check i386 images and 32-bit JNI libraries.

```
linux-x86 → gone; build linux-x64
```

16 Upgrading 21 → 25: what you gain

Two years of features land on one LTS

21→25 brings FFM, gatherers, Class-File API, unnamed variables; check Unsafe, native access, Security Manager.

```
--illegal-native-access=deny
```

Do / don't, the headline features

Compact source files & main()

Do: Write `IO.println`, not `System.out.println`, in compact files, and keep the launch command `java File.java` in a comment on the first line.

Don't: Don't put a compact file on a class path for other code to use: the implicit class has an implementation-specific name. Don't write a bare `println`; the static imports of the previews were dropped.

AOT cache ergonomics & profiles

Do: Train with the production flags and a representative workload (hit the real endpoints), and rebuild the cache whenever the JAR, the JDK or the flags change.

Don't: Don't expect the one-step form to fit a small container: cache creation runs as a sub-process with a heap like the training heap, so budget double or use the two-step workflow there.

Scoped values

Do: Keep the field `private static final`, bind at the entry point and let callees `get()`. Rebind with a nested `where` when one callee needs a different value.

Don't: Don't treat it as a mutable per-thread variable: there is no `set`, the binding ends when `run` returns, and a plain `Thread.start()` does not inherit it. `get()` on an unbound value throws.

Upgrading 21 → 25: what you gain

Do: Run the whole build on 25 with `--illegal-native-access=deny` and `--sun-misc-unsafe-memory-access=deny` to find every library that will break later, then upgrade those libraries first.

Don't: Don't enable preview flags to make an old codebase compile; preview class files are tied to one release. Don't assume a `SecurityManager` sandbox still does anything: disabled for good in 24.

Don't mix these up

ScopedValue vs. ThreadLocal

A `ThreadLocal` is a **variable**: set anywhere, copied into child threads. A `ScopedValue` is a **binding**: made by `where(...).run()`, immutable, gone when `run` returns, shared with forked subtasks.

Compact source file vs. single-file launch

Single-file launch (11) is a **launcher** feature: `java X.java` compiles in memory. A compact source file (25) is a **language** feature: no class, instance `main`, `IO`. Use both at once.

import module vs. import pkg.*

`import pkg.*` brings **one package** on demand. `import module M;` brings **every package M exports** to you, plus those of modules it requires transitively. Both lose to a single-type import.

StableValue vs. final with lazy init

A `final` field must be set **in the constructor**. Lazy init with `volatile` is late but **never constant-folded**. A `StableValue` is late **and** folded, because the JVM knows it is written at most once.

AOT cache vs. CDS archive

A CDS archive (10) holds **parsed classes**. The AOT cache (24–25) holds them **loaded and linked**, plus heap objects and, from 25, **method profiles**. `-XX:AOTCache` replaces `-XX:SharedArchiveFile`.

jdk.ExecutionSample vs. jdk.CPUTimeSample

`ExecutionSample` samples on **wall-clock** intervals, Java frames only. `CPUTimeSample` (25, experimental, Linux) samples per **CPU time consumed**, native frames included, and reports lost samples.

Generational ZGC vs. generational Shenandoah

Both low-pause and generational in 25. ZGC: **by default** since 23, every build. Shenandoah: `-XX:ShenandoahGCMode=generational`, **not in Oracle's builds**.

Preview vs. incubator vs. experimental

Preview: a complete feature behind `--enable-preview`, tied to one release. **Incubator**: an API in a `jdk.incubator.*` module, may change freely. **Experimental**: a JVM feature that may be refined or dropped.

Two years of previews graduate, Project Leyden's AOT cache becomes a one-line workflow, and the LTS badge makes 25 the version your company will actually run.

Full cards: study guide, chapter 12 · knowledge graph · quiz · practice/

13 Java 26–27 · Faster by default, ready for post-quantum · cheat sheet

Speak HTTP/3 and hybrid TLS → Start, allocate and collect faster by default → Keep previewing: lazy, structured, primitive → Warn before you remove → Evaluate a release in a week

1 NETWORKING AND SECURITY

1 HTTP/3 in the HTTP Client

QUIC under HttpClient, opt-in

Opt in: version(HTTP_3) on client or request; H3_DISCOVERY = ALT_SVC, HTTP_3_URI_ONLY, ANY. HTTP/2 is default.

```
.version(HttpClient.Version.HTTP_3)
```

2 Post-quantum hybrid TLS

X25519MLKEM768 on by default

TLS 1.3 negotiates X25519MLKEM768 by default (X25519 + ML-KEM-768); configure with jdk.tls.namedGroups.

```
-Djdk.tls.namedGroups=X25519MLKEM768
```

3 PEM API, previews two and three

Renamed twice, final in 28 PREVIEW

Preview goes on: PEM (was PEMRecord), BinaryEncodable (was DEREncodable), withFactoriesOf. Final in 28.

```
decode(text, PrivateKey.class)
```

4 JFR in-process data redaction

Secrets never reach the .jfr file

JFR masks matching env vars, properties and args as [REDACTED] in-process; FlightRecorderOptions:redact-key=.

```
redact-key=+*apikey*
```

2 FASTER BY DEFAULT

5 AOT object caching with any GC

Leyden's heap snapshot works with ZGC

AOT-cached heap objects load with any collector (ZGC included) through a streamed format; same training flags.

```
-XX:+UseZGC -XX:AOTCache=app.aot
```

6 G1 throughput: two card tables

Write barrier: 50 instructions → 12

Second card table ends barrier synchronisation; x64 barrier 50 → 12 instructions, +5–15% throughput. No flags.

```
-Xlog:gc*:file=gc.log
```

7 G1 default in all environments

Small containers get G1, not Serial

G1 is the default even with 1 CPU or under 1792 MB (was Serial). Serial still available: -XX:+UseSerialGC.

```
java -Xlog:gc -version
```

8 Compact headers by default

The 64-bit header for everyone

64-bit headers are the default; opt out with -XX:-UseCompactObjectHeaders. Up to 22% less heap.

```
-XX:-UseCompactObjectHeaders
```

3 STILL PREVIEWING

9 Lazy Constants

StableValue, renamed and pruned PREVIEW

Preview. LazyConstant.of(() -> v).get(); List/Map/Set.ofLazy(...). Was StableValue in 25; of(supplier) only.

```
LazyConstant.of(() -> build()).get()
```

10 Structured concurrency

Joiner API keeps moving; still preview

PREVIEW

Preview 6–7: anySuccessfulOrThrow, list results, ExecutionException on failure, awaitAll gone, timeout cause.

```
Joiner.<T>anySuccessfulOrThrow()
```

11 Primitive types in patterns

Stricter dominance, still preview PREVIEW

Preview 4–5: same exact-conversion matching; dominance covers primitives (long l before int i is an error).

```
case int i -> ...
```

12 Vector API

Twelve incubations, waiting for Valhalla

INCUBATOR

Incubator 11–12, API unchanged; SLEEF 3.9.0. Waits for value objects (JEP 401, preview in 28).

```
--add-modules jdk.incubator.vector
```

4 WARNINGS AND REMOVALS

13 Prepare to make final mean final

Reflection on final fields now warns

Field.set on final instance fields warns (26), denies later. Test with --illegal-final-field-mutation=deny.

```
--illegal-final-field-mutation=deny
```

14 Applet API removed

The 1996 headline feature is gone ~~REMOVED~~

java.applet, JApplet, AppletInitializer removed. Deprecated 9 → for removal 17 → gone in 26.

```
extends Applet   // gone in 26
```

15 Reading a release in a week

How to evaluate a new JDK, fast

JEP list, release notes, build with -Xlint:all, run with the deny flags, canary with JFR. A week per release.

```
javac --release 27 -Xlint:all
```

Do / don't, the headline features

HTTP/3 in the HTTP Client

Do: Start with client-level `version(HTTP_3)` and the default discovery: the client races HTTP/3 against HTTP/2 and keeps whichever connects first, so a server without h3 costs nothing. Put `response.version()` in your metrics.

Don't: Don't use `HTTP_3_URI_ONLY` against servers you don't control: the request fails instead of downgrading, and it fails whenever a proxy is in the path. Don't expect QUIC through firewalls that drop UDP 443.

Post-quantum hybrid TLS

Do: Test the handshake through every load balancer and corporate proxy in the path before rolling out; the bigger ClientHello is where things break. Keep the classical groups in the list as fallback.

Don't: Don't disable it to "keep things simple": harvest-now-decrypt-later is the threat, and browsers and large CDNs already negotiate this group. Don't look for pure ML-KEM; the JEP keeps hybrid only.

G1 default in all environments

Do: When memory is extremely tight, measure both: Serial still has the smallest footprint and stays selectable with `-XX:+UseSerialGC`. Set `-XX:MaxRAMPercentage` deliberately either way.

Don't: Don't assume a container limit under 1792 MB still means Serial when reading GC logs from 27; the ergonomics changed, and so may your pause and footprint numbers.

Compact headers by default

Do: Re-size heaps after upgrading: a service that needed 4 GB may be fine with 3.5, and in containers the saved memory is real money.

Don't: Don't carry `-XX:+UseCompactObjectHeaders` over from 25 scripts; it is a no-op now. Don't keep the opt-out flag longer than it takes to fix the tool that needed it.

Don't mix these up

HTTP/2 vs. HTTP/3

HTTP/2 multiplexes streams over **one TCP connection** (one lost packet stalls all streams) and is the **default**. HTTP/3 runs over **QUIC on UDP** with TLS 1.3 inside, streams recover independently, and you must **opt in** with `Version.HTTP_3`.

ALT_SVC vs. HTTP_3_URI_ONLY

`ALT_SVC` sends the first request over HTTP/2 and **switches** once the server advertises h3. `HTTP_3_URI_ONLY` goes straight to HTTP/3 and **fails** if the server (or a proxy) cannot. Without a hint, `ANY` races both.

Hybrid vs. pure post-quantum

Hybrid (X25519MLKEM768) combines a classical and a post-quantum exchange; both must break. **Pure** ML-KEM would trust the new algorithm alone and is a non-goal of JEP 527. ML-KEM itself arrived in 24 (JEP 496).

StableValue vs. LazyConstant

Same idea, two releases. `StableValue` (25) is created empty and filled by `orElseSet`. `LazyConstant` (26+) takes the **computing function at creation**, offers only `get()`, and moved the collections to `List/Map/Set.ofLazy`.

--illegal-final-field-mutation vs. --enable-final-field-mutation

The first sets the **policy**: `allow`, `warn` (26 default), `debug`, `deny`. The second **exempts** named modules (or `ALL-UNNAMED`) from that policy. Test with `deny`; exempt as little as possible.

Serial vs. G1 as default

Up to 26, a JVM with **one CPU or under 1792 MB** defaulted to **Serial**, everything else to G1. From 27, **G1 everywhere**. Serial is still there with `-XX:+UseSerialGC`; Parallel, ZGC and Shenandoah are unchanged.

JFR redaction vs. not recording

Redaction keeps the event (`jdk.InitialSystemProperty` still exists) but writes `[REDACTED]` for **matching keys and arguments**, inside the JVM. Disabling the event drops **all** properties, including the harmless ones you need for diagnosis.

Mappable vs. streamable AOT objects

Mappable (24–25): cached objects are laid out for one collector and mapped into the heap; fast, but G1/Serial/Parallel only. **Streamable** (26): references stored as indices, objects materialised at start-up; works with **any** collector, ZGC included.

Two quiet-looking releases that change the defaults under every application: G1 and compact headers for everyone, HTTP/3 and a post-quantum handshake on the wire, and the warnings that say what the next LTS will refuse.